Week 9 – Gaussian Mixture Models and Expectation Maximization

DS3010: Introduction to Machine Learning Lab

Timing: 02:00 PM to 04:45 PM **Max Marks:** 10

Instructions

1. Submit one `.ipynb` file containing all answers. The name should be `[student_name]_[rollno]_[lab].ipynb`
2. Write the questions in separate text blocks before the answers.
3. Outputs for all sub-questions should be given and the code should be executable.
4. Write justifications for your choices where needed.
5. Ensure that all plots include clear labels and legends for better interpretation.
6. **The EM algorithm for the Gaussian Mixture Model must be implemented from scratch.** You may use `numpy` and `matplotlib` (and `itertools` from the Python standard library), but the use of library implementations (such as `GaussianMixture` from `scikit-learn` or `multivariate_normal` from `scipy`) is strictly prohibited.
7. **Use of generative AI tools (such as ChatGPT, Gemini, etc.) is strictly prohibited.** Any submission found to contain AI-generated or plagiarized content will receive a score of zero, and disciplinary action.

Setup

Reproducibility. So that your results are the same every time the notebook is run, create the random generator **once** with `rng = np.random.default_rng(42)` and use it only for generating the data. For EM initialisation, use a **separate** generator inside your EM function (see Part 2).

Plotting helper. You may use the following function (as given) to draw the 1- and 2-standard-deviation contours of a 2-D Gaussian on a `matplotlib` axis:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Ellipse

def draw_ellipse(ax, mean, cov, color):
    """Draw the 1- and 2-standard-deviation contours of N(mean, cov)."""
    vals, vecs = np.linalg.eigh(cov) # eigenvalues in ascending order
    angle = np.degrees(np.arctan2(vecs[1, 1], vecs[0, 1]))
    for n_std in (1, 2):
        width, height = 2 * n_std * np.sqrt(vals[::-1])
        ax.add_patch(Ellipse(mean, width, height, angle=angle,
                             fill=False, edgecolor=color, linewidth=1.5))
```

Tasks

1. Data generation (2 Marks)

(a) Define three 2-dimensional Gaussian distributions with some overlap. Choose your own mixing weights π_k, means $\boldsymbol{\mu}_k$ and covariance matrices $\boldsymbol{\Sigma}_k$ $(k = 1, 2, 3)$ such that:

- the weights are not all equal, each weight is at least 0.15, the weights sum to 1, and each has at most two decimal places (for example, 0.45);
- at least one covariance matrix has a non-zero off-diagonal entry, so that its contours are tilted;
- every covariance matrix is symmetric and positive definite (check that both eigenvalues returned by `np.linalg.eigvalsh` are positive);
- the components overlap partly but can still be told apart by eye. As a guide, keep the means about 3 to 5 units apart and the diagonal entries of each $\boldsymbol{\Sigma}_k$ between 0.5 and 2.

Present your chosen parameters. These are the *true parameters* for the rest of the lab.

(b) Using `rng.multivariate_normal`, draw $900\,\pi_k$ points from component k, for $k = 1, 2, 3$ in that order. Stack them into a data matrix `X` of shape 900×2 and keep a vector `y` of the true component labels. Do not shuffle the data.

(c) Plot all 900 points, using a different colour for each true component, and draw the contours of each true Gaussian using `draw_ellipse`. The overlap between the components should be visible; if the components do not overlap at all, or overlap so much that they cannot be told apart, adjust your parameters in (a).

2. EM from scratch (4 Marks)

(a) Write a function `gaussian_pdf(X, mu, Sigma)` that returns the multivariate Gaussian density $\mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$ for every row $\mathbf{x}_n$ of `X`. Compute it for all rows at once using `numpy` array operations rather than a Python loop over the points; this keeps Part 4 fast.

(b) Write a function with the following signature that fits a K-component GMM using EM:

```
def em_gmm(X, K, seed, tol=1e-6, max_iter=500):
```

One iteration consists of one E-step (compute the responsibilities γ_{nk}) followed by one M-step (update π_k, $\boldsymbol{\mu}_k$ and $\boldsymbol{\Sigma}_k$). The function must:

- **Initialise** the means by choosing K distinct data points as follows:

```
rng = np.random.default_rng(seed)
idx = rng.choice(N, K, replace=False) # initial means are X[idx]
```

  Set every $\boldsymbol{\Sigma}_k$ to the covariance of the full dataset (`np.cov(X.T)`), and set every $\pi_k = 1/K$.
- In the M-step, add `1e-6 * np.eye(2)` to each updated covariance matrix so that it stays invertible.
- Compute the log-likelihood $\ell = \sum_n \log \sum_k \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ after initialisation and after every iteration.
- **Stop** when the change in log-likelihood between two consecutive iterations is less than `tol`, or after `max_iter` iterations.
- Return the final parameters, the final responsibilities, the list of log-likelihood values, and a copy of the parameters at iterations $0, 5, 10, \ldots$ and at the final iteration (needed for Part 2(d)).

(c) Run `em_gmm` on `X` with $K = 3$ and `seed = 42`. Report the number of iterations taken to converge and the final log-likelihood. Plot the log-likelihood against the iteration number, and verify that it never decreases.

(d) Show the progress of EM in a grid of subplots: one subplot for the initialisation (iteration 0), one after every 5 iterations (5, 10, 15, ...), and one for the final iteration. In each subplot, colour every point by its most responsible component ($\arg\max_k \gamma_{nk}$, computed from the current parameters), draw the current components with `draw_ellipse`, and show the iteration number and log-likelihood in the title.

3. Comparison with the true parameters (2 Marks)

(a) The order of the components learned by EM is arbitrary ("component 1" of your model need not correspond to true component 1). Match the learned components to the true ones by trying all $3! = 6$ orderings (`itertools.permutations`) and choosing the ordering that minimises the total Euclidean distance between the learned and true means. Use this matching for the rest of Part 3.

(b) Present a table comparing the true and estimated π_k, $\boldsymbol{\mu}_k$ and $\boldsymbol{\Sigma}_k$ for each component. Which component is estimated least accurately? Suggest a reason.

(c) Redo the plot of Part 1(c) using the learned model: colour each point by its most responsible (matched) component and draw the learned contours. Place it side by side with the plot of Part 1(c), and report the percentage of points whose assigned component equals their true component.

4. Effect of initialisation (2 Marks)

(a) Run `em_gmm` with $K = 3$ for `seed` $= 0, 1, 2, 3, 4$. Report the number of iterations and the final log-likelihood for each seed in a table. Do all runs reach the same solution? If any run ends with a lower log-likelihood, explain what has happened; if all runs agree, explain why EM can still end at a worse solution in general. Which run would you keep, and why?

(b) Summarise your observations from this lab in 4–6 sentences.

All the best for your lab! Happy coding!